

Práctica n°3. Introducción a los Sistemas Operativos

El objetivo de esta práctica es que el alumno desarrolle habilidades concernientes a Shell Scripting.

Resolución de ejercicios teóricos :

1. ¿Qué es el Shell Scripting? ¿A qué tipos de tareas están orientados los script? ¿Los scripts deben compilarse? ¿Por qué?

Shell scripting es la creación de scripts o programas en un shell de la computadora, como Bash (Usado en la cátedra), para automatizar tareas en el sistema operativo. Un **script de shell** es un archivo de texto que contiene una serie de comandos que se ejecutan en secuencia cuando el script es invocado.

Está orientado a : - Automatización de tareas repetitivas.
- Administración del sistema.
- Interacción con otros programas y herramientas, etc.

No deben compilarse los scripts ya que los scripts son interpretados por la shell en el momento que se ejecutan.

2. Investigar la funcionalidad de los comandos echo y read.

(A) ¿Cómo se indican los comentarios dentro de un script?

(B) ¿Cómo se declaran y se hace referencia a variables dentro de un script?

- ECHO: Con este comando podemos mostrar un mensaje o valor de una variable en la salida estándar (pantalla).
- READ: Con este comando leemos de pantalla una o más variables.

A. Los comentarios se indican con : #

B. Las variables de un script son de tipo string y se declaran : **var="Hola"** y para poder acceder a ellas se usa el signo \$ antes del nombre de las variables ej :
echo \$var.

4. Parametrización: ¿Cómo se acceden a los parámetros enviados al script al momento de su invocación? ¿Qué información contienen las variables \$# , \$* , \$? Y \$HOME dentro de un script?

- Los parámetros se envían al script al momento de ejecutarlos por separado, y para acceder a ellos desde el script es a través de las variables \$1,\$2..... en adelante, son guardados por orden de llegada en esas variables.
- \$# : Contiene la cantidad de parámetros que se le envió al script.
- \$* : Lista todos los parámetros que se le pasaron al script.
- \$? : Contiene el valor del último comando ejecutado.
- \$HOME : para referenciar el directorio home del usuario sin importar quién esté ejecutando el script. Esto es útil cuando necesitas acceder a archivos o directorios en el home del usuario.

5. ¿Cual es la funcionalidad del comando exit? ¿Qué valores recibe como parámetro y cuál es su significado?

El comando `exit` en un script de shell se utiliza para terminar la ejecución del script o del shell actual. Al finalizar, el comando `exit` devuelve un código de salida (o estado de salida) que indica si el script se completó con éxito (0) o si ocurrió algún error (1..255) .

Cuando se ejecuta el comando `exit` se corta la ejecución del Script por más que haya código después de él, sirve para verificar por ejemplo el buen pasaje de parámetros para que no se ejecute más si no pasaron los indicados .

6. El comando `expr` permite la evaluación de expresiones. Su sintaxis es: `expr arg1 op arg2`, donde `arg1` y `arg2` representan argumentos y `op` la operación de la expresión. Investigar que tipo de operaciones se pueden utilizar.

Operaciones Matemáticas: Suma (+), resta (-), multiplicación(*), división(/) y módulo(%).

Operaciones Lógicas: Comparaciones de igualdad, diferencia, menor que, mayor que, menor o igual que, mayor o igual que.

Otra opción también es encerrar la operación en : `$(( op ))`

7. El comando “test expresión” permite evaluar expresiones y generar un valor de retorno, true o false. Este comando puede ser reemplazado por el uso de corchetes de la siguiente manera [expresión]. Investigar qué tipo de expresiones pueden ser usadas con el comando test. Tenga en cuenta operaciones para:

evaluación de archivos, evaluación de cadenas de caracteres y evaluaciones numéricas.

- **Evaluación de Archivos:** Verificar existencia, tipo y permisos de archivos y directorios.
- **Evaluación de Cadenas:** Comparar cadenas, y verificar si están vacías o no.
- **Evaluación Numérica:** Comparar valores numéricos usando operadores aritméticos.

EJEMPLO :

-e archivo : Verifica si existe el archivo → `test -e archivo` o `[ -e archivo ]`.

```
if [ -e "archivo.txt" ];  
then  
    echo "El archivo existe."  
else  
    echo "El archivo no existe."  
fi
```

El comando **test** y el uso de corchetes son herramientas esenciales en scripts de shell para realizar verificaciones y tomar decisiones basadas en la evaluación de expresiones.

8. Estructuras de control. Investigue la sintaxis de las siguientes estructuras de control incluidas en shell scripting:

- if
- case
- while
- for

- select

Selección de alternativas:

Decisión:

```
if [ condition ]
then
    block
fi
```

Selección:

```
case $variable in
    "valor 1")
        block
    ;;
    "valor 2")
        block
    ;;
    *)
        block
    ;;
esac
```

Menú de opciones:

```
select variable in opcion1 opcion2 opcion3
do
    # en $variable está el valor elegido
    block
done
```

- **C-style:**

```
for ((i=0; i < 10; i++))
do
    block
done
```

- **Con lista de valores (foreach):**

```
for i in value1 value2 value3 valueN;
do
    block
done
```

while

```
while [ condition ] #Mientras se cumpla la condición
do
    block
done
```

until

```
until [ condition ] #Mientras NO se cumpla la condición
do
    block
done
```

9. ¿Qué acciones realizan las sentencias break y continue dentro de un bucle? ¿Qué parámetros reciben?

- Break [n] corta la ejecución de n niveles de loops. (Si no se especifica un número, **break** sale solo del bucle más interno).
- Continue [n] salta a la siguiente iteración del enésimo loop que contiene esta instrucción. (Si no se especifica un número, **continue** afecta solo al bucle más interno).

10. ¿Qué tipo de variables existen? ¿Es shell script fuertemente tipado? ¿Se pueden definir arreglos? ¿Cómo?

11. ¿Pueden definirse funciones dentro de un script? ¿Cómo? ¿Cómo se maneja el pasaje de parámetros de una función a la otra?

15. Comando cut. El comando cut nos permite procesar la líneas de la entrada que reciba (archivo, entrada estándar, resultado de otro comando, etc) y cortar columnas o campos, siendo posible

indicar cuál es el delimitador de las mismas. Investigue los parámetros que puede recibir este comando y cite ejemplos de uso.